

DB Tables

```
class RawPost(Base):
    __tablename__ = "raw_posts"
    id = Column(Integer, autoincrement=True, primary_key=True)
    platform = Column(String)
    source = Column(String)
    title = Column(String)
    full_text = Column(Text)
    language = Column(String)
    created_date = Column(DateTime(timezone=True))
    week_start_date = Column(DateTime(timezone=True))
    source_url = Column(String, nullable=True)
    post_topic = relationship("PostTopic", back_populates="raw_post", uselist=False)
    __table_args__ = (UniqueConstraint("platform", "source", "title", "created_date"),)
```

```
class PostTopic(Base):
    __tablename__ = "posts_with_topics"
    id = Column(Integer, autoincrement=True, primary_key=True)
    raw_post_id = Column(Integer, ForeignKey("raw_posts.id"))
    platform = Column(String)
    source = Column(String)
    title = Column(String)
    full_text = Column(Text)
    week_start_date = Column(DateTime(timezone=True))
    topic_id = Column(Integer)
    topic_label = Column(String)
    food_category = Column(String)
    raw_post = relationship("RawPost", back_populates="post_topic")
```

```
class WeeklySnapshot(Base):
    __tablename__ = "weekly_snapshots"
    id = Column(Integer, autoincrement=True, primary_key=True)
    food_category = Column(String, nullable=False)
    topic_label = Column(String, nullable=True)
    week_start_date = Column(DateTime(timezone=True), nullable=False)

    growth_rate = Column(Float, nullable=False)
    platform_count = Column(Integer, nullable=False, default=0)
    sustained_growth = Column(Integer, nullable=False, default=0)
    ratio_to_peak = Column(Float, nullable=False)
    rolling_avg = Column(Float, nullable=False)
    ra4 = Column(Float, nullable=False)
    rank_wk = Column(Float, nullable=False)

    #non feature columns for future predictions
    total_posts = Column(Integer, nullable=True)
    future_avg = Column(Float, nullable=True)
    future_growth_rate = Column(Float, nullable=True)
    # Target
    will_trend = Column(Boolean, nullable=True)
    __table_args__ = (UniqueConstraint("food_category", "week_start_date"),)
```

```
class TrendPrediction(Base):
    __tablename__ = "trend_predictions"
    id = Column(Integer, autoincrement=True, primary_key=True)
    food_category = Column(String)
    topic_label = Column(String)

    #features
    growth_rate = Column(Float, nullable=False)
    platform_count = Column(Integer, nullable=False, default=0)
    sustained_growth = Column(Integer, nullable=False, default=0)
    ratio_to_peak = Column(Float, nullable=False)
    rolling_avg = Column(Float, nullable=False)
    ra4 = Column(Float, nullable=False)
    rank_wk = Column(Float, nullable=False)

    #features
    trend_probability = Column(Float)
    will_trend = Column(Boolean)
    prediction_week = Column(DateTime(timezone=True))
```

Load packages

Fetch Data from YouTube

Fetch Data from Google trends

Fetch Data from APIFY

1. Find the Lanugages used (Lingua)
2. CreateDate is post created date mostly available from Data
3. WeekstartDate is calculated using createdDate (Monday is considered as week start day)
4. Map and merge the data in RawPost Format and save to DB

LOAD EXISTING RAW POSTS FROM DATABASE

Generate Clean_Text (input for berTopic):
1. Clean title
2. Clean full_text
3. Cleaned text and title cobined and stored to Clean_combine_text

Keeps only Clean_combine_text with at least 5 words.
Reason: Very small texts create poor clusters.

Sentence Embeddings
Every Full_Text becomes a numerical vector.

Calculate Dynamic Parameters:
n_neighbors = min(20, max(8, n_docs // 15))
n_components = min(10, max(5, n_docs // 50))
min_cluster_size = 10
min_samples = 3

Parameter	Algorithm	Purpose	Recommendation
'n_neighbors'	UMAP	Controls how many neighboring documents are considered during dimensionality reduction.	""Keep Dynamic"" (or a small fixed value like 5 if datasets are similar)
'n_components'	UMAP	Controls the number of dimensions after reduction.	""Keep Dynamic"" (or a small fixed value like 5 if datasets are similar)
'min_cluster_size'	HDBSCAN	Minimum number of documents required to form a topic.	""Tune and Fix""
'min_samples'	HDBSCAN	Controls how strict the clustering is and how many outliers are produced.	""Tune and Fix""

BERTopic Model Creation and Topic Discovery

After calculating the dynamic clustering parameters, the supporting models required for topic modeling are created and configured. These components include:

Sentence Transformer (MPNet) → Generates semantic embeddings (numerical vector representations) for each preprocessed document.

UMAP → Reduces the high-dimensional embedding vectors into a lower-dimensional space while preserving their semantic relationships, making clustering more efficient.

HDBSCAN → Groups semantically similar documents into clusters and automatically determines the number of topics while identifying outlier documents.

CountVecorizer → Creates a vocabulary of important words and n-grams from the clustered documents after removing stop words and filtering very common or very rare terms.

Class-TF-IDF (c-TF-IDF) → Calculates the importance of words within each cluster to identify the most representative keywords for every discovered topic.

Representation Models (Maximal Marginal Relevance, KeyBERTInspired, and Part-of-Speech Filtering) → Refine the topic representation by removing redundant keywords, selecting semantically meaningful terms, and retaining informative words such as nouns and adjectives.

After configuring these supporting components, a BERTopic model instance is created by passing them as parameters to the BERTopic() constructor. And after the prediction it will provide Topic and Probablity of prediction is correct.

Topic have: Topic id, Name (topicid, topicname), representation_Docs(Full_text)
Important: Probability is not consider any where...

Reduce Outlier:In this area topic -1 is considered to outlier we try to match those with any topic find using strategy="c-tf-idf", then also not matched with topic remains -1

Based on topic keyword (representation) from BerTopic Model we just do the a LLM call to set the food_category and is_valid_category returned by LLM.
Based on this group by categoryName and another column topic_ids which can be have multiple topic_ids of of that damne categoryName

Combine the result from BerTopic model and LLM and save to DB(PostTopic table)

LOAD EXISTING POSTS with Topic FROM DATABASE

Extract Features and Lables based on the equation:
growth_rate = Column(Float, nullable=False)
platform_count = Column(Integer, nullable=False, default=0)
sustained_growth = Column(Integer, nullable=False, default=0)
ratio_to_peak = Column(Float, nullable=False)
rolling_avg = Column(Float, nullable=False)
ra4 = Column(Float, nullable=False)
rank_wk = Column(Float, nullable=False)

#non feature columns for future predictions
total_posts = Column(Integer, nullable=True)
futures_avg = Column(Float, nullable=True)

Generate Weekly snapshot and save to DB

Load Weekly snapshot from DB

Train the model Using Expanding window with weekly snapshot data

Build and deploy final model and predict
Save prediction result to DB